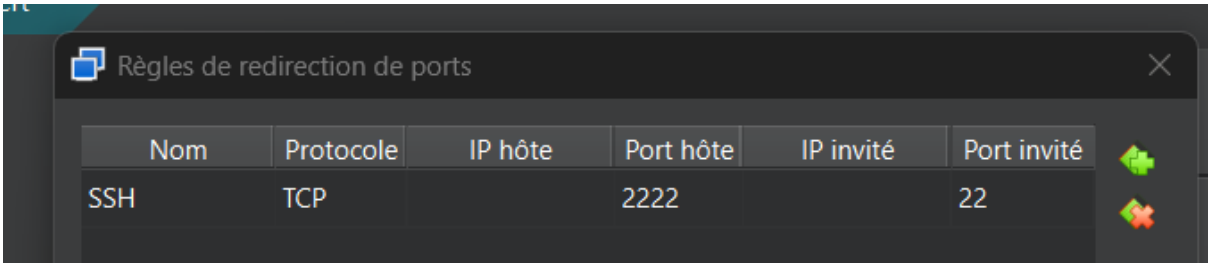
CONFIGURATION VM

Pour activer le copier coller, j'utilise mon terminal windows:
on passe en root: sudo -i
on va passer par le ssh: apt install -y openssh-server
puis en configuration:




22 étant le port pour le ssh, et pour ne pas créer de conflit avec mon ordinateur on donne 2222.

Et pour activer notre port ssh:
on regarde si il est actif: systemctl status ssh
et on l'active: systemctl start ssh && systemctl enable ssh
puis dans powershell, mon terminal windows, pour me connecter:
ssh -p 2222 toto@localhost (toto qui est le nom de ma vm)

J'obtiens les informations sur l'ip de ma vm: ip a, me donne enp0s3 10.0.2.15/24
C'est ce que l'on va vouloir surveiller avec l'IDS

INSTALLATION DE L'IDS
Puis installation de snort échoué:

add-apt-repository universe -y
add-apt-repository multiverse -y
apt update

sauf que
apt install -y snort
apt install -y snort3

ne fonctionnait pas je ne trouvais pas snort.

Donc je suis passé à suricata:

apt install -y suricata
puis pour le mettre à jour : suricata-update

CONFIGURATION IDS

On configure le fichier : nano  /etc/suricata/suricata.yaml
en modifiant l'interface qu'il écoute pour qu'il écoute bien notre enp0s3 récupéré précédemment et non eth0.
Et on ajoute : - /etc/suricata/rules/local.rules
dans la section rules-files, pour avoir un fichier de règles que suricata va appliquer.

Donc on crée ce fichier et on écrit une règle:
pour créer le fichier : touch /etc/suricata/rules/local.rules
echo 'alert icmp any any -> any any (msg:"[ALERTE LOCALE] Ping ICMP detecte sur enp0s3"; sid:1000001; rev:1;)' > /etc/suricata/rules/local.rules

La règle signifie que on fait une alerte (un log) en cas de protocole icmp (ping) détécté provenant de n'importe quelle adresse Ip et port, en direction de n'importe quelle adresse Ip et port également (mais comme on surveille notre interface des pings, ça va surveiller pour notre ip), puis on écrit notre message affiché, l'identifiant de la règle, et la version de la règle.

et on redémarre le système en l'activant:
systemctl enable suricata
systemctl restart suricata
systemctl status suricata

Test:
On va sur un autre terminal faire : tail -f /var/log/suricata/fast.log
Ce qui affiche les 10 dernières lignes du fichier de log, et le -f permet de garder le fichier ouvert, de suivre ses changements en temps réel.

Puis on ping: ping -c 3 127.0.0.1 fonctionne
mais rien n'est détecté
Tandis que depuis un autre terminal : ping -c 3 8.8.8.8
fonctionne et est détécté.
Le ping 8.8.8.8 qui envoie des paquets sur une adresse qui sert à savoir si on a internet, et comme ça part de notre vm, on utilise la carte réseau virtuelle enp0s3, et suricata à été configuré pour l'écouter. il a vu passer le paquet au moment précis où il franchissait la carte pour sortir vers Internet et revenir. La règle a matché, et l'alerte a été générée instantanément.

```
10/05/2026-22:44:35.782234  [**] [1:1000001:1] [ALERTE LOCALE] Ping ICMP detecte sur enp0s3 [**] [Classification: (null)
] [Priority: 3] {ICMP} 10.0.2.15:8 -> 8.8.8.8:0
10/05/2026-22:44:35.802429  [**] [1:1000001:1] [ALERTE LOCALE] Ping ICMP detecte sur enp0s3 [**] [Classification: (null)
] [Priority: 3] {ICMP} 8.8.8.8:0 -> 10.0.2.15:0
```

Le premier ping ne fonctionne pas car il se ping soi-même mais en local, on ne passe pas par notre interface enp0s3, donc ce n'est pas surveillé, tandis que l'autre ping lui est visible dessus.

5 ATTAQUES:

Exploitation d’une vulnérabilité web

Ce cas simule une attaque ciblée contre une application web. L'attaquant manipule l'URL dans le but d'exécuter un bout de code. Suricata analyse la requête HTTP GET et déclenche une alerte s'il repère la chaîne de caractères suspecte dans l'URL.

C’est le scénario utilisé dans le sujet, et les applications web sont l'un des vecteurs d'attaque les plus courants et les plus exposés sur Internet. Ici, on n’analyse pas seulement l’adresse IP, mais aussi le contenu du paquet.

Donc pour détecter l’attaque on va faire regarder si le contenu du trafic http contient “exploit=1” et seulement dans l’url avec http_uri

alert http any any -> any any (msg:"[SCENARIO 1] Tentative Web - Parametre exploit detecte"; content:"exploit=1"; http_uri; sid:1000002; rev:1;)

et pour tester l’attaque : curl "http://example.com/?exploit=1"

Fuites de données

Ce cas simule une tentative de connexion via un protocole non chiffré, donc qui exposerait potentiellement nos informations si utilisé. L'IDS détecte le protocole, ici le FTP (port 21). L'alerte est déclenchée dès le premier paquet de synchronisation (TCP SYN) envoyé vers ce port, avant même que l'échange de données n'ait lieu.

Les protocoles non chiffrés transmettent les identifiants et les données en clair sur le réseau, exposant les mots de passe par simple écoute du réseau.

Donc pour détecter l’attaque on va regarder le port pour savoir quel protocole est utilisé, et on ajoute le flag S pour le tcp qui déclenche justement dès le premier paquet.

alert tcp any any -> any 21 (msg:"[SCENARIO 2] Securite - Tentative de connexion FTP (Non chiffre)"; flags:S; sid:1000003; rev:1;)

Pour tester l’attaque : curl telnet://example.com:21

Cheval de Troie

Ce scénario surveille le trafic qui sort de notre réseau. L'objectif est de détecter un ordinateur interne qui tente de communiquer vers l'extérieur sur le port 4444. Ce port est très connu car c'est celui utilisé par défaut par un outil de piratage très populaire pour contrôler des machines à distance, ce qui en fait un trafic suspect. On suppose qu'un pirate a déjà réussi à rentrer (par exemple via du phishing) et a installé un virus (cheval de Troie). Le virus va essayer de contacter le serveur du pirate pour recevoir ses ordres.

Ce scénario montre qu'on ne protège pas que l'entrée, on peut aussi surveiller les machines déjà infectées.

Donc pour détecter l’attaque on va regarder encore une fois le port et dès le premier paquets, on alerte.

alert tcp any any -> any 4444 (msg:"[SCENARIO 3] Compromission - Trafic suspect vers port backdoor Metasploit (4444)"; flags:S; sid:1000004; rev:1;)

Pour tester l’attaque : curl telnet://example.com:4444

Phase de reconnaissance avec scan réseau

Ce cas simule le repérage fait avant une attaque. Ce scénario détecte l'utilisation d'outils automatiques de scan en lisant la signature de l'outil cachée dans les requêtes web (l'en-tête "User-Agent"). L'IDS repère ici la signature de l'outil "Nmap", un scanner qui toque à toutes les portes du serveur pour voir ce qui est ouvert.

La reconnaissance est la toute première étape d'une attaque. En détectant le pirate à ce stade, l'administrateur peut bloquer l'adresse IP du pirate avant même qu'il n'ait pu tenter quoi que ce soit. C'est une défense proactive.

Donc pour détecter l’attaque on va regarder, si dans le trafic web “Nmap”, qui est un logiciel de piratage connu, apparaît dans l’en-tête avec http_header.

alert http any any -> any any (msg:"[SCENARIO 4] Reconnaissance - Scanner Nmap detecte"; content:"Nmap"; http_header; sid:1000005; rev:1;)

Pour tester l’attaque : curl -A "Nmap/7.92" "http://example.com"

DOS

Ce cas simule une attaque où l'on veut faire planter un serveur. L'IDS est configuré pour alerter si on reçoit un paquet réseau de test (un ping) qui est anormalement gros et lourd (plus de 1000 octets). Le but d'un pirate ici n'est pas de voler des données, mais d'inonder notre système avec des paquets géants pour le surcharger, provoquant un ralentissement ou un crash.

Ce scénario montre aussi que l'IDS peut repérer des anomalies physiques sur la taille des données, et pas seulement en lisant du texte.

Donc pour détecter l’attaque on va regarder si on reçoit des ping d’une certaine taille avec dsize>1000, car un ping normal fait 64 octets.

alert icmp any any -> any any (msg:"[SCENARIO 5] Attaque DoS - Paquet Ping geant detecte"; dsize:>1000; sid:1000006; rev:1;)

Pour tester l’attaque : ping -s 1500 -c 1 8.8.8.8

SYSLOG-NG

On installe : apt install -y syslog-ng
Puis on configure avec un fichier que l’on crée : nano /etc/syslog-ng/conf.d/suricata-elk.conf
On y met :

```
source s_suricata {
   file("/var/log/suricata/eve.json" flags(no-parse));
};

# 2. LA DESTINATION : On envoie vers la base de données Elasticsearch
destination d_elasticsearch {
   elasticsearch-http(
      index("suricata-logs")
      url("http://127.0.0.1:9200/_bulk")
      template("${MESSAGE}\n")
   );
};

log {
   source(s_suricata);
   destination(d_elasticsearch);
};
```

Donc nous avons la source que syslog va examiner, avec le json, qui n’est pas ce que je regardais, car c’était très verbeux et pour vérifier simplement le fonctionnement, c’était moins pratique. Et on indique no-parse, car le fichier ici est déjà du json.
Puis On indique la destination qui va être ElasticSeach, avec l’index qui signifie le nom de l’index de logs, donc on précise que ça vient de suricata, puis l’url qui ici correspond à sa propre adresse (en interne sans passer enp0s3, donc suricata) et au port 9200 qui est le standard pour ElasticSearch.
Enfin on lie les deux.

Puis on redémarre :
systemctl enable syslog-ng
systemctl restart syslog-ng

ELASTICSEARCH

pour installer ElasticSearch et qu'il reste même la machine éteinte, il faut faire ça:
# 1. Télécharger la clé de sécurité d'Elastic
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg

# 2. Ajouter l'adresse du magasin (dépôt)
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | tee /etc/apt/sources.list.d/elastic-8.x.list

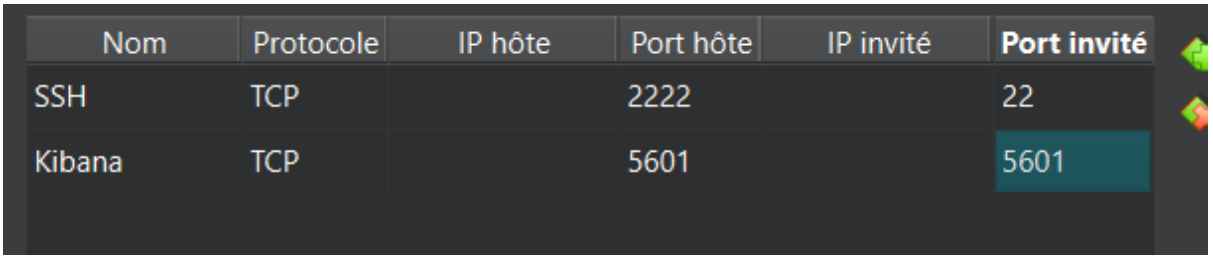
apt update
apt install -y elasticsearch kibana

Puis on change la configuration de ElasticSearch:
nano /etc/elasticsearch/elasticsearch.yml
et on met:
xpack.security.enabled en false

Cela va nous permettre de déposer nos paquets syslog-ng facilement.
Puis on change la config de Kibana pour pouvoir l'ouvrir avec Windows, car je suis sur le powershell en ssh pour pouvoir faire des copier coller.

Avec :
nano /etc/kibana/kibana.yml
et on décomment la ligne de server.host, on met "0.0.0.0" à la place de localhost pour pouvoir y accéder depuis ma machine sous windows.

Et comme pour ssh, on ouvre un port pour Kibana,  pour y accéder.



| Nom | Protocole | IP hôte | Port hôte | IP invité | Port invité |
|---|---|---|---|---|---|
| SSH | TCP | | 2222 | | 22 |
| Kibana | TCP | | 5601 | | 5601 |

Enfin :
systemctl enable elasticsearch kibana
systemctl start elasticsearch kibana

puis quand je vais à http://localhost:5601, j'obtiens un site elastic, qui est j'imagine la page d'accueil sans configuration.

Mais je ne peux pas créer de data view, car je n'ai pas encore de données dans ma base de données ElasticSearch. Il faut donc faire une attaque pour que mon Suricata la détecte, qu'elle passe à syslog puis à ElasticSearch.

Pour savoir si la connexion se fait on utilise:
curl 'http://127.0.0.1:9200/_cat/indices/suricata-logs?v'
Et si suricata-logs apparaît, ce qui est l'index que j'avais mis.

MAIS il semblerait que j'ai un problème entre mon syslog et ElasticSearch, car la liaison ne se fait pas, je n'ai pas suricata-logs qui apparaît, à cause d'un problème apparemment connu sur les versions syslog 4.8.1.
Et pour le corriger, comme je ne peux pas mettre à jour mon syslog, je dois changer sa configuration :
nano /usr/share/syslog-ng/include/scl/elasticsearch/elastic-http.conf
et supprimer la mention de index._type ='type'
J'ai aussi supprimé type("")

Maintenant, ça fonctionne.

Pour être vraiment sûr j'ai testé toutes les simulations d'attaques puis fait :

```
curl -s -X POST 'http://127.0.0.1:9200/suricata-logs/_search?pretty' \
  -H 'Content-Type: application/json' \
  -d '{
    "size": 10,
    "query": {
      "match": {
        "event_type": "alert"
      }
    },
    "sort": [{"timestamp": {"order": "desc"}}],
    "_source": [
      "timestamp",
      "event_type",
      "src_ip",
      "dest_ip",
      "alert"
    ]
  }'
```

Ce qui va nous donner uniquement nos alertes si elles sont bien enregistrés, de la plus récente à la moins récente pour 10 alertes.

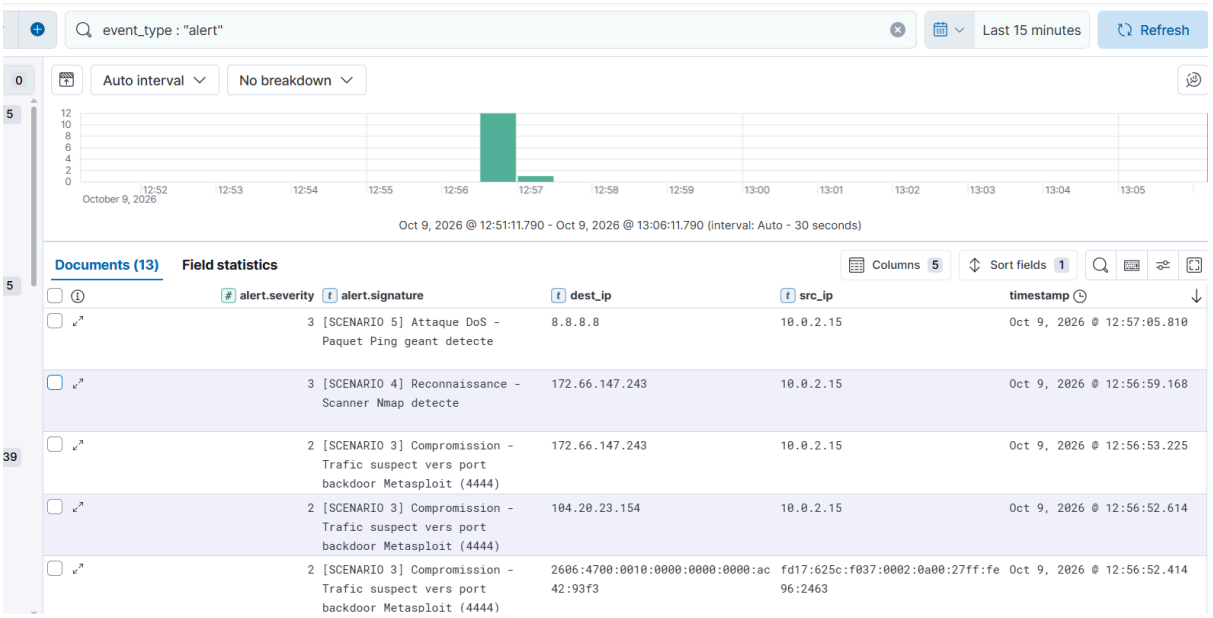
Pour bien visualiser les alertes sur Kibana et pour trier nos alertes, j'ai rajouté la classification et la priorité:

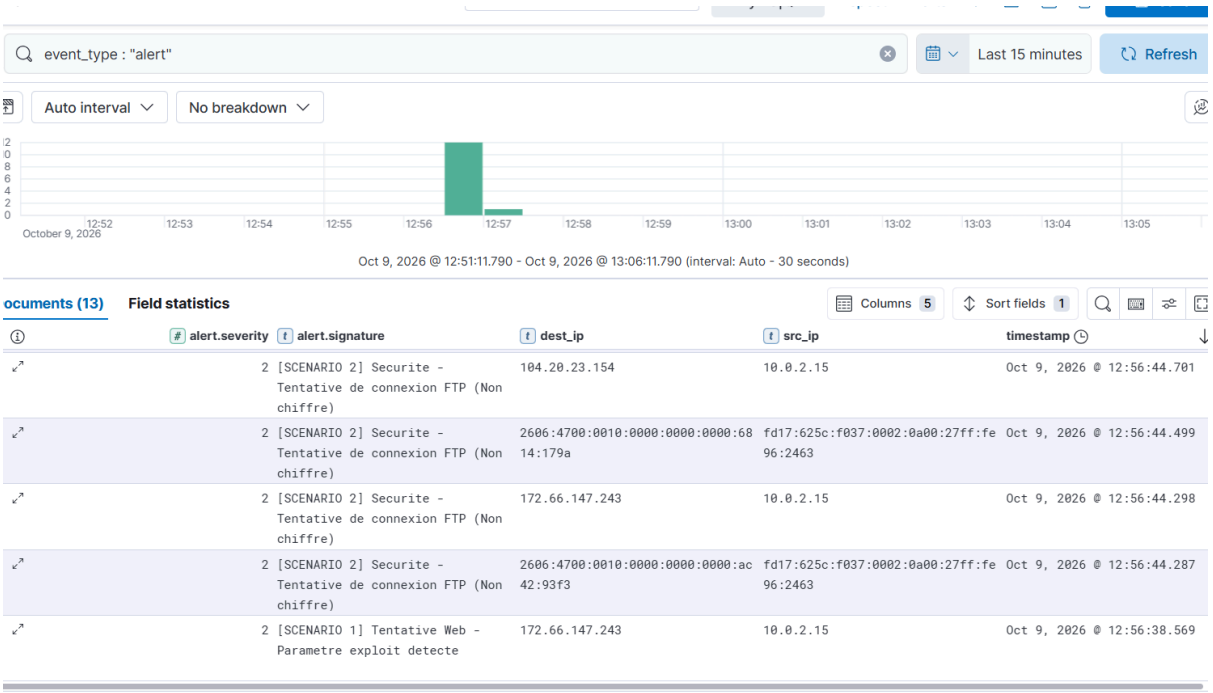
alert http any any -> any any (msg:"[SCENARIO 1] Tentative Web - Parametre exploit detecte"; content:"exploit=1"; http_uri; classtype:web-application-activity; priority:2; sid:1000002; rev:1;)
alert tcp any any -> any 21 (msg:"[SCENARIO 2] Securite - Tentative de connexion FTP (Non chiffre)"; flags:S; classtype:bad-unknown; priority:2; sid:1000003; rev:1;)
alert tcp any any -> any 4444 (msg:"[SCENARIO 3] Compromission - Trafic suspect vers port backdoor Metasploit (4444)"; flags:S; classtype:bad-unknown; priority:2; sid:1000004; rev:1;)

alert http any any -> any any (msg:"[SCENARIO 4] Reconnaissance - Scanner Nmap detecte"; content:"Nmap"; http_header; classtype:network-scan; priority:3; sid:1000005; rev:1;)
alert icmp any any -> any any (msg:"[SCENARIO 5] Attaque DoS - Paquet Ping geant detecte"; dsize:>1000; classtype:attempted-dos; priority:3; sid:1000006; rev:1;)

Avec pour la 1, qui correspond à une potentielle tentative d'exploitation de vulnérabilité, donc à étudier car danger possible, donc en priorité 2 et la catégorie adaptée.
Pour la 2, nous avons une utilisation d'un protocole en clair, ce qui peut être un risque pour la confidentialité, donc en priorité 2, mais pour la catégorie, comme nous ne sommes pas certain d'une fuite, on indique bad-unknown, pour signifier le potentiel danger non identifié.
Pour la 3,  nous avons un potentiel cheval de Troie, mais en se basant juste sur l'utilisation d'un port, donc pareil, danger possible, donc même priorité car il faut étudier la situation et la catégorie est la même.
Pour la 4, nous avons potentiellement un scan de réseau, ce qui n'est pas encore l'attaque, donc en priorité 3, avec la catégorie adaptée.
Pour la 5, nous avons un possible déni de service, mais comme on ne se base que sur 1 ping lourd, on met la priorité en 3, car c'est une investigation moins urgente que les autres, dans le sens ou un ping ne suffit pas à alerter d'un déni de service.

Maintenant on obtient sur Kibana:






Avec donc la priorité configurée, le message configuré, l'adresse ip qui est visé par l'attaque, l'adresse ip qui est la source de l'attaque, et la date.

Ce projet permet donc de détecter des "attaques" avec des alertes, mais ça ne reste qu'une légère introduction, car ici on n'attrape pas de réels attaques, mais simplement des simulations définies, qui sont donc faciles à détecter.
En temps normal, il faudrait des règles pour chaque attaque possibles et qui soient plus précises, ce qui pourrait peut-être être automatisé.
Ici nous avons des détections de pings lourds par exemple, mais il faudrait une détection d'une suite de pings lourds, avec un seuil pour l'annoncer.
Notre projet manque de précision, donc il crée beaucoup de faux positifs ou faux négatifs, puis pour l'instant nous ne bloquons rien, c'est une simple détection, donc on pourrait avoir un IPS plutôt qu'un IDS.
D'autant plus que nous ne faisons rien des informations récoltées, comme l'ip de l'émetteur de l'attaque.

